

INTRODUCCIÓN SQL

CONSULTAS

1. INTRODUCCIÓN

El lenguaje SQL (Structured Query Language) es una herramienta para **organizar, gestionar y recuperar datos almacenados en una base de datos relacional**, por tanto, permite la comunicación con el sistema de gestión de la base de datos. Es tan conocido en bases de datos relacionales que muchos lenguajes de programación incorporan sentencias SQL como parte de su repertorio. Entre las principales características del SQL:

- Es un lenguaje **para todo tipo de usuarios** (administradores, desarrolladores y usuarios normales).
- El usuario que emplea SQL **especifica qué quiere**, no dónde ni cómo.
- Permite **hacer cualquier consulta** de datos.
- Es posible manejarlo para **consultas, actualizaciones, definición de datos y control**.

2. TIPOS DE SENTENCIAS SQL

El lenguaje SQL proporciona un gran repertorio de sentencias que se utilizan en variadas tareas, como consultar datos de la base de datos, crear, actualizar y eliminar objetos de la base de datos, crear, actualizar y eliminar datos de los objetos, controlar el acceso a la base de datos y a los objetos.

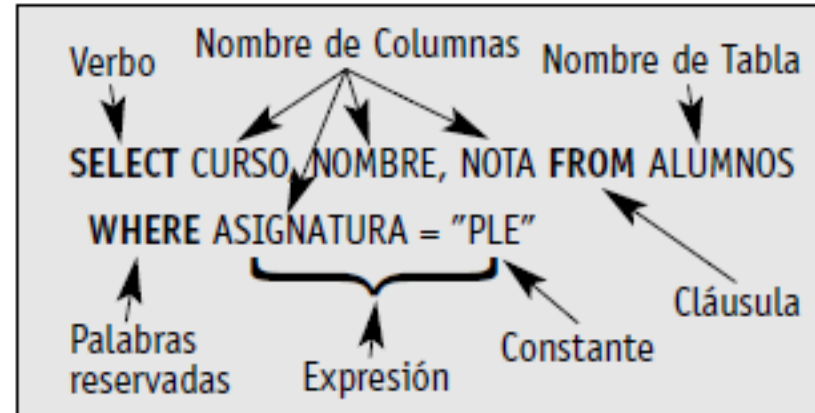
➤ Dependiendo de las tareas, podemos clasificar las sentencias SQL en varios tipos.

En la siguiente tabla se exponen las sentencias más importantes y las usadas con mayor frecuencia:

SENTENCIA		DESCRIPCIÓN
DML	<i>Manipulación de datos</i>	
	SELECT	Recupera datos de la base de datos.
	INSERT	Añade nuevas filas de datos a la base de datos.
	DELETE	Suprime filas de datos de la base de datos.
	UPDATE	Modifica datos existentes en la base de datos.
DDL	<i>Definición de datos</i>	
	CREATE TABLE	Añade una nueva tabla a la base de datos.
	DROP TABLE*	Suprime una tabla de la base de datos.
	ALTER TABLE*	Modifica la estructura de una tabla existente.
	CREATE VIEW*	Añade una nueva vista a la base de datos.
	DROP VIEW*	Suprime una vista de la base de datos.
	CREATE INDEX*	Construye un índice para una columna.
	DROP INDEX*	Suprime el índice para una columna.
	CREATE SYNONYM*	Define un alias para un nombre de tabla.
	DROP SYNONYM*	Suprime un alias para un nombre de tabla.
DCL	<i>Control de acceso</i>	
	GRANT	Concede privilegios de acceso a usuarios.
	REVOKE	Suprime privilegios de acceso a usuarios.
	<i>Control de transacciones</i>	
	COMMIT	Finaliza la transacción actual.
	ROLLBACK	Aborta la transacción actual.
	<i>SQL Programático</i>	
	DECLARE	Define un cursor para una consulta.
	OPEN	Abre un cursor para recuperar resultados de consulta.
	FETCH	Recupera una fila de resultados de consulta.
	CLOSE	Cierra un cursor.

* No existían en el estándar SQL1

3. COMPONENTES SINTÁCTICOS DE UNA SENTENCIA SQL



1. Todas **comienzan con un verbo**, que es una palabra clave que describe lo que hace la sentencia (por ejemplo, SELECT, INSERT, UPDATE, CREATE).
2. A continuación, **le siguen una o más cláusulas** que especifican los datos con los que opera la sentencia. Estas también comienzan con una palabra clave como WHERE o FROM. Algunas son opcionales y otras obligatorias. Muchas contienen nombres de tablas o de columnas, palabras reservadas, constantes o expresiones adicionales.

4. CÓMO GESTIONA EL SGBD UNA SENTENCIA

Para procesar una sentencia SQL el sistema gestor de base de datos recorre una serie de pasos:

1. **Analiza la sentencia.** Comprueba que está sintácticamente bien escrita.
2. **Valida la sentencia.** Comprueba la sentencia semánticamente. Es decir, comprueba si las tablas y las columnas referenciadas en la sentencia existen, si el usuario tiene privilegios para realizar esa operación sobre la tabla,
3. **Optimiza la sentencia.** El sistema gestor de base de datos explora la forma de llevar a cabo la ejecución de la sentencia.
4. **Genera un plan de aplicación para la sentencia.** Se genera código ejecutable.
5. **Ejecuta el plan de aplicación.**

El análisis de la sentencia no requiere acceso a la base de datos y, por tanto, suele realizarse rápidamente. Sin embargo, la optimización es un proceso muy intensivo de CPU y precisa acceder a la base de datos.

5. CONSULTA DE LOS DATOS

Para recuperar información, para realizar consultas a la base de datos, utilizaremos **una única sentencia SELECT**.

El usuario emplea esta sentencia con el **nivel de complejidad** apropiado para él: especifica qué es lo que quiere obtener (no dónde ni cómo).

De la consulta se puede obtener: cualquier unidad de datos, todos los datos, cualquier subconjunto de datos, cualquier conjunto de subconjuntos de datos.

A. SENTENCIA SELECT

```
SELECT  [ ALL | DISTINCT ]  
[ expre_colum1, expre_colum2, ..., expre_column | * ]  
FROM [ nombre_tabla1, nombre_tabla2, ..., nombre_tablan ]  
[WHERE condición]  
[ ORDER BY expr_colum [ DESC | ASC ] [ , expre_colum [ DESC | ASC ] ] ... ];
```

- Donde **expre_colum** puede ser una columna de una tabla, una constante, una expresión aritmética, una función o varias funciones anidadas.
- Todas las sentencias SQL terminan en punto y coma.

B. CLÁUSULAS DEL SELECT

La única **cláusula** de la sentencia SELECT que es **obligatoria** es la cláusula **FROM**, el resto son opcionales (no importa si se usan mayúsculas o minúsculas):

FROM: FROM [nombre_tabla1, nombre_tabla2, ..., nombre_tablan]

Especifica la tabla o lista de tablas de las que se recuperarán los datos.

```
SELECT NOM_ALUM, NOTA FROM ALUMNO;
```

Si el usuario que hace la consulta no es el propietario de la tabla, es necesario especificar el nombre de usuario delante de ella: *nombre_usuario.nombre_tabla*. Por ejemplo, si el usuario propietario de la tabla se llama PROFESOR emplearemos:

```
SELECT NOM_ALUM, NOTA FROM PROFESOR.ALUMNOS;
```

Es posible asociar un nuevo nombre a las tablas mediante **alias**. Por ejemplo, si a la tabla ALUMNOS le damos el nombre de A, las columnas de la tabla irán acompañadas de A:

```
SELECT A.NOM_ALUM A.NOTA FROM ALUMNO A;
```

B. CLÁUSULAS DEL SELECT

WHERE: [WHERE condición]

Obtiene las filas que cumplen la condición expresada. La **complejidad** de la condición es prácticamente **ilimitada**. El formato de la condición es: ***expresión operador expresión***.

Las expresiones pueden ser una constante, una expresión aritmética, un valor nulo o un nombre de columna.

➤ Operadores de comparación:

=, >, <, >=, <=, !=, <>

IN, NOT IN, BETWEEN, NOT BETWEEN

LIKE

Se pueden construir condiciones múltiples usando los **operadores lógicos** booleanos estándares: **AND**, **OR** y **NOT**. Está permitido **emplear paréntesis** para forzar el orden de evaluación.

WHERE NOTA = 5

WHERE (NOTA >10) AND (CURSO = 1)

WHERE (NOTA IS NULL) OR (UPPER (NOM_ALUM) = 'PEDRO')

B. CLÁUSULAS DEL SELECT

ORDER BY: [ORDER BY expr_column [DESC | ASC] [, expr_column [DESC | ASC]] ...]

Esta cláusula especifica el **criterio de clasificación** del resultado de la consulta. ASC especifica una ordenación ascendente, y DESC descendente.

- Puede contener expresiones con valores de columnas:

```
SELECT * FROM ALUMNOS ORDER BY NOTA/5;
```

- Es posible anidar los criterios. El situado más a la izquierda será el principal:

```
SELECT * FROM ALUMNOS ORDER BY NOM_ALUM, CURSO DESC;
```

- También se puede indicar mediante un número, que indica la posición de la columna a la derecha de SELECT:

```
SELECT DEPT_NO, DNOMBRE, LOC FROM DEPART ORDERT BY 2;
```

B. CLÁUSULAS DEL SELETC

ALL: con la cláusula ALL recuperamos todas las filas, aunque algunas estén repetidas. Es la opción por omisión.

DISTINCT: solo recupera las filas que son distintas. Z

Por ejemplo, consultamos los departamentos (columna DEPT_NO) de la tabla EMPLE:

```
SELECT DEPT_NO FROM EMPLE;
```

DEPT_NO	
20	
30	
30	
20	
30	
30	
10	
20	
10	
30	
20	
30	
20	
10	
14 filas seleccionadas.	

Aparecen todas las filas de la tabla EMPLE donde la columna DEPT_NO no sea nula; también aparecen números de departamento repetidos. Con DISTINCT se eliminan las filas repetidas:

```
SELECT DISTINCT DEPT_NO FROM EMPLE;
```

DEPT_NO
10
20
30

C. SELECCIÓN DE COLUMNAS

El formato de SELECT que nos permite seleccionar las columnas de una tabla es:

SELECT [ALL | DISTINCT]

[expre_colum1, expre_colum2, ..., expre_colum | *]

FROM [nombre_tabla1, nombre_tabla2, ..., nombre_tablan]

Dato: con la **orden DESC**ribe, obtenemos un resumen de la tabla y de sus columnas:

DESC EMPLE;

Nombre	¿Nulo?	Tipo
-----	-----	-----
EMP_NO	NOT NULL	NUMBER (4)
APELLIDO		VARCHAR2 (10)
OFICIO		VARCHAR2 (10)
DIR		NUMBER (4)
FECHA_ALT		DATE
SALARIO		NUMBER (7)
COMISION		NUMBER (7)
DEPT_NO	NOT NULL	NUMBER (2)

DESC DEPARTS;

Nombre	¿Nulo?	Tipo
-----	-----	-----
DEPT_NO	NOT NULL	NUMBER (2)
DNOMBRE		VARCHAR2 (14)
LOC		VARCHAR2 (14)

C. SELECCIÓN DE COLUMNAS

Selección de todas las columnas. Podemos recuperar las filas de dos formas:

1. Ponemos los nombres de todas las columnas, uno tras otro, separados por comas:

```
SELECT EMP_NO, APELLIDO, OFICIO, DIR, FECHA_ALT, SALARIO, COMISION,  
DEPT_NO  
FROM EMPLE;
```

2. Ponemos * que representa a todas las columnas de la tabla:

```
SELECT * FROM EMPLE;
```

Para seleccionar determinadas columnas, solo se ponen los nombres de las columnas que nos interesan. Por ejemplo, para escoger el nombre (DNOMBRE) y el número de departamento (DEPT_NO) de la tabla DEPARTS emplearemos:

```
SELECT DNOMBRE, DEPT_NO FROM DEPARTS;
```

DNOMBRE	DEPT_NO
-----	-----
CONTABILIDAD	10
INVESTIGACION	20
VENTAS	30
PRODUCCION	40

D. SELECCIÓN POR FILA

Para seleccionar determinadas filas necesitamos emplear la **cláusula WHERE** en la sentencia SEELCT, [WHERE condición].

Ejercicio 1.

Ejercicio 2.

Ejercicio 3.

E. CREAR Y UTILIZAR ALIAS DE COLUMNA.

Cuando se consulta la base de datos, los nombres de las columnas se usan como cabeceras de presentación. Si el nombre resulta demasiado largo, corto o críptico, existe la posibilidad de cambiarlo con la misma sentencia SQL de consulta creando un ALIAS.

El ALIAS se pone entre comillas dobles, a la derecha de la columna:

```
SELECT DIR "Dirección" FROM EMPLEADOS;
```

Ejercicio 4.

6. OPERADORES ARITMÉTICOS

Los operadores aritméticos sirven para formar expresiones con constantes, valores de columnas y funciones de valores de columnas. Son los que se observan en la siguiente tabla:

Operador aritmético	Operación
+	Suma
-	Resta
*	Multiplicación
/	División

Una forma posible de utilizar los operadores puede ser:

```
SELECT col1*col2, col1-col2  
FROM tabla1  
WHERE col1+col2 = 34;
```

Ejercicio 5.

7. OPERADORES DE COMPARACIÓN Y LÓGICOS.

DE COMPARACIÓN	
Operador	Función
=	Igual a
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que
!= <>	Distinto de

LÓGICOS	
Operador	Función
AND	Devuelve el valor TRUE cuando las dos condiciones son verdaderas.
OR	Devuelve el valor TRUE cuando una de las dos condiciones es verdadera.
NOT	Devuelve el valor TRUE si la condición es falsa.

Ejercicio 6.

8. CADENAS DE CARACTERES: OPERADORES DE COMPARACIÓN Y DE CONCATENACIÓN.

Para comparar cadenas de caracteres, hasta ahora hemos utilizado el operador de comparación Igual a (=). Así, a partir de la tabla EMPLE, obtenemos el apellido de los ANALISTAS del departamento 10:

```
SELECT APELLIDO  
FROM EMPLE  
WHERE OFICIO = 'ANALISTA' AND DEPT_NO=10;
```

Pero este operador no nos sirve si queremos hacer consultas de este tipo:

- Obtener los datos de los empleados cuyo apellido empiece por una 'P'.
- Obtener los nombres de alumnos que incluyan la palabra 'Pérez'.

8. CADENAS DE CARACTERES: OPERADORES DE COMPARACIÓN Y DE CONCATENACIÓN.

Para especificar este tipo de consultas, en SQL usamos el **operador LIKE** que permite utilizar los siguientes caracteres especiales en las cadenas de comparación:

- % comodín: representa cualquier cadena de 0 o más caracteres.
- _ marcador de posición: representa un carácter cualquiera.

En la cláusula WHERE este operador se utiliza:

```
WHERE columna LIKE 'caracteres_especiales'
```

En una sentencia WHERE **se pueden usar varias cláusulas LIKE anidadas por operadores AND/OR:**

```
WHERE col1 LIKE 'car_especiales' AND|OR col2 LIKE 'car_especiales'
```

8. CADENAS DE CARACTERES: OPERADORES DE COMPARACIÓN Y DE CONCATENACIÓN.

Ejemplos de uso de la cláusula LIKE:

- LIKE 'Director' la cadena 'Director'
- LIKE 'M%' cualquier cadena que empiece por 'M'
- LIKE '%X%' cualquier cadena que contenga una 'X'
- LIKE '___M' cualquier cadena de 3 caracteres terminada en 'M'
- LIKE 'N_' una cadena de 2 caracteres que empiece por 'N'
- LIKE '_R%' cualquier cadena cuyo segundo carácter sea una 'R'

Hemos de tener en cuenta que:

- las **mayúsculas y minúsculas** son significativas ('m' no es lo 'M'), y
- las constantes alfanuméricas deben encerrarse siempre entre **comillas simples**.

8. CADENAS DE CARACTERES: OPERADORES DE COMPARACIÓN Y DE CONCATENACIÓN.

Puede ser también, que en alguna determinada consulta interese **concatenar** distintas columnas cuyo tipo de dato sean cadenas de caracteres, para mostrar su contenido de manera conjunta.

Por ejemplo, tener en una tabla el primer y segundo apellido en columnas diferentes y querer mostrarlos juntos. Para ello se utiliza el **operador de concatenación** (||):

```
SELECT NOMBRE, APELLIDO1 || ' ' || APELLIDO2 AS APELLIDOS  
FROM EMPLEADOS;
```

Ejercicio 7.

Ejercicio 8.

9. NULL Y NOT NULL.

Se dice que una columna de una fila es NULL si está completamente vacía.

Para comprobar si el valor de una columna es nulo empleamos la expresión: *columna IS NULL*.

Si queremos saber si el valor de una columna no es nulo, usamos la expresión: *columna IS NOT NULL*.

Cuando comparamos con valores nulos o no nulos no podemos utilizar los operadores de igualdad, mayor o menor.

Por ejemplo, a partir de la tabla EMPLE, consultamos los apellidos de los empleados cuya comisión es nula:

```
SELECT APELLIDO FROM EMPLE WHERE COMISION IS NULL;
```

Si queremos consultar los apellidos de los empleados cuya comisión no sea nula:

```
SELECT APELLIDO FROM EMPLE WHERE COMISION IS NOT NULL;
```

10. COMPROBACIONES CON CONJUNTOS DE VALORES.

Hasta ahora, todas las comprobaciones lógicas que hemos visto comparan una columna o expresión con un valor. Por ejemplo:

```
OFICIO = 'ANALISTA' AND DEPT_NO=10.
```

Pero también podemos comparar una columna o una expresión con una lista de valores utilizando los operadores **IN** y **BETWEEN**.

10. COMPROBACIONES CON CONJUNTOS DE VALORES.

A. OPERADOR IN

El operador IN nos permite comprobar si una expresión pertenece o no (NOT) a un conjunto de valores, haciendo posible la realización de comparaciones múltiples.

Su formato es:

`<expresión> [NOT] IN (lista de valores separados por comas)`

Ejercicio 9.

Ejercicio 10.

B. OPERADOR BETWEEN.

El operador BETWEEN comprueba si un valor está comprendido o no (NOT) dentro de un rango de valores, desde un valor inicial a un valor final.

Su formato es:

`<expresión> [NOT] BETWEEN
valor_inicial AND valor_final`

11. COMBINACIÓN DE OPERADORES AND Y OR.

Los operadores AND y OR se pueden combinar de forma ilimitada, pero hay que tener cuidado al usarlos y utilizar paréntesis para agrupar aquellas expresiones que se deseen evaluar juntas; si no nos servimos de los paréntesis, es posible que los resultados no sean los deseados.

El orden de prioridad de los operadores lógicos es el siguiente:

1. el primero NOT,
2. después AND y,
3. por último, OR.

Para alterar el orden se utilizan paréntesis.

12. COMBINACIÓN DE OPERADORES.

Orden de todos los operadores que podemos usar/combinar:

1. Se evalúa la multiplicación (*) y la división (/) al mismo nivel.
2. Sumas (+) y restas (-).
3. Concatenación (| |).
4. Todas las comparaciones (<, >, ...).
5. Después evaluaremos los operadores IS NULL, IN NOT NULL, LIKE, BETWEEN.
6. NOT.
7. AND.
8. OR.

Si quisiéramos variar este orden necesitaríamos utilizar paréntesis.

Ejercicio 11.

13. SUBCONSULTAS.

A veces, para realizar alguna operación de consulta, **necesitamos los datos devueltos por otra consulta**; así, si queremos obtener los datos de los empleados que tengan el mismo oficio que 'PEPE', hemos de averiguar el oficio de 'PEPE' (primera consulta). Una vez conocido este dato, podemos averiguar qué empleados tienen el mismo oficio que 'PEPE' (segunda consulta).

Este problema se puede resolver usando una **subconsulta**, que no es más que **una sentencia SELECT dentro de otra SELECT**; consiste en **incluir** una declaración **SELECT como parte de una cláusula WHERE**. Formato:

```
SELECT ...  
FROM ...  
WHERE columna operador_comparativo (SELECT ...  
                                     FROM ...  
                                     WHERE ... );
```

La **subconsulta** se ejecuta primero y, posteriormente, el valor extraído es «introducido» en la consulta principal.

Ejercicio 12.

Ejercicio 13.

A. CONDICIONES DE BÚSQUEDA EN SUBCONSULTAS.

Las subconsultas normalmente aparecen como parte de la condición de búsqueda de una cláusula WHERE o HAVING. Las condiciones de búsqueda que nos podemos encontrar en una subconsulta son:

➤ **Test de comparación en subconsultas (>, <, <>, <=, >=, =).** Compara el valor de una expresión con un valor único producido por una subconsulta.

- Ejemplo: Obtener aquellos apellidos de empleados cuyo oficio es igual al oficio de 'GIL':

```
SELECT APELLIDO FROM EMPLA WHERE OFICIO = (SELECT OFICIO  
FROM EMPLA WHERE APELLIDO ='GIL');
```

A. CONDICIONES DE BÚSQUEDA EN SUBCONSULTAS.

➤ **Test de pertenencia a un conjunto devuelto por una subconsulta (IN).** Comprueba si el valor de una expresión coincide con uno del conjunto de valores producido por una subconsulta.

- Ejemplo: Obtener aquellos apellidos de empleados cuyo oficio sea alguno de los oficios que hay en el departamento 20:

```
SELECT APELLIDO FROM EMPL  
WHERE OFICIO IN (SELECT OFICIO  
                 FROM EMPL WHERE DEPT_NO=20);
```

A. CONDICIONES DE BÚSQUEDA EN SUBCONSULTAS.

➤ **Test de existencia (EXISTS , NOT EXISTS).** Examina si una subconsulta produce alguna fila de resultados. El test es TRUE si devuelve filas, si no es FALSE.

- Ejemplo: listar los departamentos que tengan empleados:

```
SELECT DNOMBRE, DEPT_NO  
FROM DEPART  
WHERE EXISTS (SELECT *  
                FROM EMPL  
                WHERE EMPL.DEPT_NO= DEPART.DEPT_NO);
```

Para calcular los que no tengan empleados se usa el operador **NOT EXISTS**.

A. CONDICIONES DE BÚSQUEDA EN SUBCONSULTAS.

➤ **Test de comparación cuantificada (ANY y ALL).** Se usan en conjunción con los operadores de comparación (>, <, <>, <=, >=, =).

- **ANY:** compara el valor de una expresión con cada uno del conjunto de valores producido por una subconsulta, si alguna de las comparaciones individuales da como resultado TRUE, ANY devuelve TRUE, si la subconsulta no devuelve nada devolverá FALSE.

- Ejemplo: obtener los datos de los empleados cuyo salario sea igual a algún salario de los empleados del departamento 30:

```
SELECT * FROM EMPLE WHERE SALARIO = ANY (SELECT SALARIO  
FROM EMPLE WHERE DEPT_NO=30);
```

- **ALL:** compara el valor de una expresión con cada uno del conjunto de valores producido por una subconsulta, si todas las comparaciones individuales dan como resultado TRUE, ALL devuelve TRUE, en caso contrario devuelve FALSE.

- Ejemplo: Obtener los datos de los empleados cuyo salario sea menor a cualquier salario de los empleados del departamento 30:

```
SELECT * FROM EMPLE WHERE SALARIO < ALL (SELECT SALARIO  
FROM EMPLE WHERE DEPT_NO=30);
```

B. SUBCONSULTAS QUE GENERAN VALORES SIMPLES.

Subconsultas que devuelven una fila o un valor simple; en éstas se utilizan los **operadores de comparación** (>, <, <>, <=, >=, =).

Si la subconsulta obtiene más de una fila, se produce un mensaje de error.

- Por ejemplo, con la siguiente consulta se pretende obtener los apellidos de los empleados cuyo oficio coincida con algún oficio del departamento 20:

```
SELECT APELLIDO FROM EMPL WHERE OFICIO =  
(SELECT OFICIO FROM EMPL WHERE DEPT_NO=20);
```

ERROR: ORA-01427: la subconsulta de una sola fila devuelve más de una fila

En el departamento 20 hay varios oficios por tanto la subconsulta devuelve varias filas. Por ello, **cuando una subconsulta devuelve más de una fila no se puede recurrir a los operadores de comparación.**

C. SUBCONSULTAS QUE GENERAN LISTAS DE VALORES.

Subconsultas que **devuelven más de una fila o más de un valor**. Cuando una subconsulta devuelva más de un valor, usaremos el **operador IN en la cláusula WHERE**.

- La solución al ejemplo anterior sería la siguiente:

```
SELECT APELLIDO FROM EMPLE WHERE OFICIO IN  
(SELECT OFICIO FROM EMPLE WHERE DEPT_NO=20);
```

El tipo de dato de la expresión situada después de WHERE debe coincidir con el tipo de dato devuelto por la subconsulta.

Ejercicio 14.

Ejercicio 15.

D. SUBCONSULTAS CORRELACIONADAS.

Subconsulta correlacionada es aquella que **hace referencia a una columna o varias de la consulta más externa**. A veces la subconsulta hace uso de columnas que tienen el mismo nombre que las columnas de las tablas usadas en la consulta más externa. Si la subconsulta necesita acceder a esas columnas deberá definirse un alias en la tabla más externa.

- Por ejemplo, deseamos obtener los datos de los empleados cuyo salario sea el máximo salario de su departamento:

```
SELECT * FROM EMPLE E WHERE SALARIO =  
(SELECT MAX(SALARIO) FROM EMPLE WHERE DEPT_NO = E.DEPT_NO);
```

La subconsulta devuelve para cada fila que se recupere de la consulta más externa el máximo salario del departamento que se está recuperando en la consulta externa; para referenciar a dicho departamento se necesita el alias E usado en la tabla de la consulta externa.

14. COMBINACIÓN DE TABLAS

Hasta ahora, en las consultas que hemos realizado solo se ha utilizado una tabla, indicada a la derecha de la palabra FROM; pero hay veces que una consulta necesita columnas de varias tablas. En este caso, las tablas se expresarán a la derecha de la palabra FROM.

Sintaxis general:

```
SELECT columnas de las tablas citadas en la cláusula "from"  
FROM tabla1, tabla2, ...  
WHERE tabla1.columna = tabla2.columna;
```

14. COMBINACIÓN DE TABLAS

Cuando combinamos varias tablas, hemos de tener en cuenta una serie de reglas:

- Es posible **unir tantas tablas como deseemos**.
- En la cláusula **SELECT** se pueden citar **columnas de todas las tablas**.
- **Si hay columnas con el mismo nombre** en las distintas tablas de la cláusula FROM, se deben identificar, especificando **NombreTabla.NombreColumna**.
- Si el nombre de una columna existe solo en una tabla, no será necesario especificarla como NombreTabla.NombreColumna. Sin embargo, hacerlo mejoraría la legibilidad de la sentencia SELECT.
- El **criterio** que se siga **para combinar las tablas ha de especificarse en la cláusula WHERE**. Si se omite esta cláusula, que especifica la condición de combinación, el resultado será un PRODUCTO CARTESIANO, que emparejará todas las filas de una tabla con cada fila de la otra.

Ejercicio 16.

Ejercicio 17.